

ChaosProbe MSc Thesis Advisory Report

Executive summary

This can absolutely become a defensible MSc systems thesis, but only if you stop trying to sell it as a universal placement theorem and instead frame it as a **fault-class-specific, measurement-aware empirical study of Kubernetes resilience** supported by a substantial systems artifact. The public repository already looks far beyond a toy project: it exposes a Python CLI, an explicit placement engine with six non-default strategies, LitmusChaos orchestration, Prometheus/Kubernetes/Locust/Litmus metric collection, Neo4j storage, statistical comparison commands, provenance capture, data-quality checks, example summaries, tests, and reproduction-oriented documentation. That gives you a very real artifact contribution. It also gives you a credible basis for a thesis if the manuscript stays disciplined about scope and evidence. ¹

The strongest thesis is **not** “I found the best placement strategy.” It is closer to: **placement effects under chaos are fault-class-specific and measurement-layer-specific; on small Kubernetes clusters, pod-delete faults can be dominated by service-network churn rather than node-level contention, which makes aggregate resilience scores unstable and shifts the reliable signal to mechanism-level metrics and route-level user outcomes.** That framing aligns with Kubernetes’ own behavior around topology spread, Service/EndpointSlice convergence, terminating endpoints, kube-proxy sync, conntrack state, CPU throttling, and node-pressure eviction. It also positions ChaosProbe cleanly against LitmusChaos and Chaos Mesh, which inject faults but do not by themselves provide your controlled placement mutation plus cross-layer comparative analysis. ²

The public repo’s most defensible existing headline is methodological, not heroic: your own defense script says that after the east-west probe fix and filtering to trustworthy churn runs, the aggregate score does **not** reproduce well enough to rank strategies, while mechanism-level metrics do. Your reproduction doc similarly states that the aggregate mean resilience score is not expected to produce a stable strategy ordering, and that the mechanism metrics are the real reproducibility bar. That is not a weakness if you present it correctly. It is a solid systems/measurement finding. It becomes weak only when you overclaim “refutation” or “best strategy.” ³

If you want the **best fast thesis**, cut the strategy space down to **default, colocate, spread**, keep the artifact, and run **two fault classes** with enough repetitions: **pod-delete** as the churn case and **node-memory-hog** as the contention case. That is the cleanest way to test your real hypothesis: that the ordering of placements changes with fault class, and that different measurement layers carry the stable signal. Pod-level CPU hog is a weak headline experiment because Kubernetes enforces CPU limits by throttling, which can transform your “contention” fault into self-throttling noise. Node-level memory pressure is much more likely to produce a strong placement-dependent result because kubelet eviction, OOM behavior, and node pressure act at the node scope where placement actually matters. ⁴

My blunt ranking is this. **Best next experiment:** node-memory-hog. **Best use of your current evidence:** pod-delete as a churn-mechanism study. **Best ambitious extension:** targeted network latency or loss on

high-value service edges, especially if you can show that dependency-aware or colocated placements reduce exposure to degraded cross-node paths. **Weak or boring choices:** pod-cpu-hog as a central claim, Redis latency without topology-aware targeting, and disk I/O stress on Online Boutique unless you first prove the workload is actually disk-sensitive. ⁵

Thesis defensibility and framing

The thesis is worthy if you present it as a **measurement study plus artifact thesis**, not as a grand scheduler paper. That is already supported by the repository's visible structure. ChaosProbe defines explicit non-default placements including colocate, spread, random, adversarial, best-fit, and dependency-aware; records intended-versus-actual placement match; captures recovery as deletion-to-scheduled and scheduled-to-ready phases; exposes bootstrap confidence intervals and Holm-corrected pairwise Mann-Whitney comparisons; records run metadata including git commit, Kubernetes version, runtime, and CNI hint; and ships a `doctor` command that flags tainted iterations, low placement match, OOM kills, missing recovery, inconclusive confidence intervals, and provenance gaps. Those are exactly the kinds of features that make an artifact academically respectable. ⁶

The strongest defensible framing is this:

Recommendation	Proposed wording
Primary research question	<i>How do Kubernetes pod-placement strategies interact with fault class, and which measurement layers provide stable evidence of resilience differences under chaos experiments?</i>
Core subquestion	<i>Does pod-delete primarily manifest as service-network churn rather than resource contention in this experimental regime?</i>
Contribution statement	<i>ChaosProbe is a placement-aware chaos-evaluation framework for Kubernetes that enables controlled placement mutation, fault injection, cross-layer metric collection, statistical comparison, and topology-aware data storage; using it, this thesis shows that placement effects are fault-class-specific and measurement-layer-specific, and that aggregate resilience scores can fail to provide stable rankings even when mechanism metrics do.</i>
Best thesis title	<i>Fault-Class-Specific Placement Effects in Kubernetes Chaos Experiments</i>
Safer alternate title	<i>A Measurement Study of Placement-Aware Chaos Engineering in Kubernetes with ChaosProbe</i>

Why this framing works: Kubernetes' official guidance on pod topology spread is about balancing pods across failure domains for availability and utilization, not about universal superiority under every fault model. Kubernetes' own service networking stack also makes pod termination an asynchronous, multi-layer event that involves EndpointSlice state, kube-proxy rule convergence, and connection-tracking behavior. Your thesis can therefore ask a fair systems question: when does the "spread is safer" intuition transfer, and when does it not? ⁷

The framing to avoid is the one visible in some prose in the repo and slide script: “the literature is refuted,” “spread is disproven,” “recovery time predicts nothing,” or “the best strategy is X.” Your own public docs already give you a safer path: the tool warns that n below 8 per group is underpowered, the reproduction guide says the aggregate score is not expected to reproduce a stable ranking, and the slide script itself narrows trustworthy conclusions to post-fix churn runs and mechanism metrics. That is strong evidence for **local, conditional, measurement-aware claims** and poor support for **broad refutation language**. 8

My recommendation is to split your contributions explicitly:

Contribution type	What you can honestly claim
Novel research contribution	In this setup, pod-delete behaves more like a churn/routing reconvergence fault than a node-contention fault; therefore placement effects are fault-class-specific, and mechanism metrics can diverge from aggregate score rankings.
Engineering contribution	ChaosProbe: placement mutator, Litmus runner, cross-layer metric collection, Neo4j topology store, statistical CLI, provenance capture, and data-quality diagnostics.
Replication / validation contribution	Reproducing and operationalizing Kubernetes service-networking behavior documented in official docs and issues, especially around kube-proxy synchronization, EndpointSlice termination semantics, and contrack-related disruption under endpoint churn.
Pilot contribution	Early pod-cpu-hog observations suggesting confounding by CPU limits and motivating stronger node-level contention faults.

Literature positioning and related systems

ChaosProbe sits at the intersection of four literatures: cluster scheduling and placement, chaos engineering, Kubernetes internals, and microservice resilience measurement. The scheduling literature gives you the intuition that placement matters. Borg and the Borg/Omega/Kubernetes lineage frame practical cluster management tradeoffs; Sparrow gives randomized low-latency scheduling intuition; Quasar and related interference-aware work motivate performance-aware placement; and Medea is especially relevant because it says precise control of container placement matters for performance and resilience, including whether to colocate or separate services. But none of those papers are running LitmusChaos faults on a Kubernetes microservice benchmark while comparing cross-layer outcomes under controlled placement interventions. 9

The chaos-engineering literature gives you the right methodological home, but not your exact question. Basiri et al. define chaos engineering at a high level; Netflix’s Simian Army is the historical root of failure injection by killing instances; LitmusChaos and Chaos Mesh provide broad fault libraries and orchestration/workflow support; MicroRes is the closest methodological relative because it argues that resilience profiling depends on how degradation disseminates from system metrics to user-aware metrics; and ChaosEater is a recent automation paper that is orthogonal rather than competitive. ChaosProbe’s novelty is not “I can inject chaos into Kubernetes.” That is commodity now. Your novelty is **controlled placement mutation plus multi-layer comparative measurement**. 10

Kubernetes internals are where your mechanism story becomes credible. Official docs state that topology spread constraints balance pods across nodes or other topology domains for availability and utilization. Official Service and EndpointSlice docs state that kube-proxy watches Services and EndpointSlices and synchronizes forwarding state on each node. Official EndpointSlice docs also state that terminating endpoints are exposed explicitly and are usually ignored for normal traffic, with special handling in some cases. The kube-proxy conntrack configuration exists because tracked NAT flows are part of the implementation. Kubernetes’ own documentation on NodeLocal DNSCache explicitly says it avoids kube-proxy DNAT rules and connection tracking for DNS. All of that supports your claim that pod deletion can create a **reconvergence problem** rather than just a **compute contention problem**. ⁷

The nearest “competitor” papers do not kill your thesis. Mutiny! is about Kubernetes failures driven by datastore corruption and shows that even single faults can propagate to cluster-wide consequences, but it is not a placement study. The recent cloud-edge resilience-evaluation work runs a massive campaign of failure injection scenarios, but it does not compare placement strategies or separate churn-based from contention-based placement effects. Network-aware microservice-placement modeling is close conceptually, but it is modeling and optimization, not controlled empirical chaos injection on Kubernetes. That means ChaosProbe is best positioned as **complementary empirical evidence** that tests how placement theories survive contact with real Kubernetes failure mechanisms. ¹¹

The workload choice is fine for an MSc thesis. Google’s Online Boutique is an 11-microservice polyglot application with a Locust load generator and clearly defined service interactions; DeathStarBench independently argues that microservices create networking, operating-system, cluster-management, and tail-latency challenges that traditional monolith benchmarks do not capture; and *The Tail at Scale* remains the canonical warning that means can hide the user-visible story in distributed systems. That is exactly why your aggregate score versus mechanism-metric separation is a credible thesis angle. ¹²

A compact positioning comparison is below. It synthesizes the official docs, platform docs, and primary papers above. ¹³

System or paper	What it does	Overlap with ChaosProbe	What ChaosProbe can claim relative to it
Kubernetes topology spread	Native spreading for HA/utilization	Placement policy baseline	Tests whether spread intuition transfers under specific fault classes
kube-proxy / EndpointSlice / conntrack	Service routing and endpoint convergence machinery	Mechanism layer under pod churn	Provides experimental measurement of how those internals appear in application outcomes
LitmusChaos	Fault injection platform, ChaosCenter, ChaosHub	Fault execution substrate	Adds controlled placement mutation, statistical comparison, provenance, and graph storage

System or paper	What it does	Overlap with ChaosProbe	What ChaosProbe can claim relative to it
Chaos Mesh	Rich fault catalog and workflows	Alternative fault platform	Same story: ChaosProbe is about evaluation design, not raw fault variety
Borg / Quasar / Medea / Sparrow	Scheduling and placement literature	Motivation for placement relevance	Tests placement intuitions under chaos rather than designing a new scheduler
MicroRes	Resilience profiling via degradation dissemination	Closest methodological cousin	Extends the “metric layers matter” argument to placement-aware chaos studies
Mutiny!	K8s failure analysis via data-store fault injection	Broad resilience relevance	Different layer; not placement-aware
Cloud-edge failure-injection study	Massive K8s resilience dataset	Broad comparative resilience study	Adds placement as the manipulated factor
DeathStarBench / Online Boutique	Microservice benchmark context	Benchmark/workload choice	Reasonable, representative microservice target
Tail-at-scale literature	Tail-latency methodology	Measurement philosophy	Justifies emphasizing p95/p99 and route-level effects over means alone

Ranked experiments and hypotheses

The public repo currently defaults to eight strategies and often five iterations, but the same repo also documents that pairwise comparisons are underpowered for n below 8 and that the aggregate score is noisy. So the first hard recommendation is: **cut breadth, buy depth**. For a thesis defense, more repetitions on fewer strategies will beat a wide but underpowered matrix every time. ¹⁴

Experiment ranking

This ranking is based on novelty, defensibility, feasibility on your documented small cluster, expected signal strength, literature alignment, and risk of producing boring or uninterpretable results. It is grounded in your repo's current setup and in Kubernetes/Litmus behavior. ¹⁵

Experiment	Novelty	Defensibility	Feasibility	Expected signal	Risk	Verdict
Node-memory-hog × placement	High	High	Medium	High	Medium	Best new core experiment

Experiment	Novelty	Defensibility	Feasibility	Expected signal	Risk	Verdict
Pod-delete × placement with mechanism metrics	Medium	High	High	Medium on score, high on mechanism	Medium	Best use of existing evidence
Targeted network latency/loss on service edge	High	Medium	Medium	High if scoped well	Medium-high	Best ambitious extension
Node-cpu-hog × placement	Medium	Medium	Medium	Medium	Medium	Good backup contention experiment
Redis latency	Medium	Medium	Medium	Medium if topology-aware, low otherwise	Medium	Only if you can target specific dependency edges
Pod-cpu-hog × placement	Low-medium	Low-medium	High	Low-medium	High	Weak headline; keep as pilot or appendix
Disk I/O stress / disk-fill	Low	Low	Medium	Low on Online Boutique	High	Avoid unless pilot proves disk sensitivity

Recommended minimal and ambitious sets

The **single best minimal thesis set** is:

Element	Recommendation
Strategies	<code>default</code> , <code>colocate</code> , <code>spread</code> only
Faults	<code>pod-delete</code> and <code>node-memory-hog</code>
Primary metrics	pod-delete: conntrack flush proxy, TCP retransmits, route p99, error rate, d2s/s2r split; node-memory-hog: memory pressure, OOM/evictions, route p99, error rate, probe pass rate
Secondary metrics	aggregate resilience score, harmonic or lower-tail score variants, CPU throttling/usage

Element	Recommendation
Repetitions	minimum 8 valid iterations per strategy per fault ; target 10
Random seeds	not needed in the core claim set
Outcome	a clean two-fault story: churn case versus contention case

The **stronger ambitious set** is:

Element	Recommendation
Strategies	<code>default</code> , <code>colocate</code> , <code>spread</code> , <code>dependency-aware</code> ; keep <code>random</code> only as appendix if you have five seeds
Faults	add <code>pod-network-latency</code> or <code>pod-network-loss</code> with destination scoping
Analysis	strategy × fault interaction using an aligned-rank or mixed-effects analysis, plus route-level breakdowns
Extension	one limited multi-replica variant for the target service to test whether the churn story survives partial failover
Outcome	a genuinely strong “fault-class-specific placement interaction” thesis

Detailed experiment designs

The table below gives the top four designs in the format an examiner would expect. The repo itself already supports many of the needed fields and statistics; where I recommend broader analyses, that is because the thesis question is multi-factor. ¹⁶

Experiment	Hypothesis	Independent variables	Dependent variables	Controls	Expected mechanism	Metrics to collect
Node-memory-hog	Denser placement will worsen memory-pressure effects and user-visible degradation relative to spread under node-level contention.	Placement strategy; optionally memory-hog intensity after pilot tuning	p95/p99 latency, error rate, probe success, OOM kills, evictions, MemoryPressure duration, recovery split	Fixed load, same target service, same duration, same requests/limits, no autoscaling, same cluster batch	Node-level memory pressure affects all colocated workloads and can trigger reactive OOM plus kubelet eviction	node memory availability, PSI if available, MemoryPressure events, OOM count, eviction count, Locust route stats, probe verdicts

Experiment	Hypothesis	Independent variables	Dependent variables	Controls	Expected mechanism	Metrics to collect
Pod-delete churn	Spread increases transport/network reconvergence disruption relative to colocate under forced single-replica deletion, even if aggregate scores remain unstable.	Placement strategy	conntrack-flush proxy, retransmits, kube-proxy/network-programming metrics if available, route p99, error rate, d2s/s2r	Identical pod-delete YAML, same target, same load, same scrape setup, same warmup	Deletion changes EndpointSlice and kube-proxy state asynchronously; remote service paths are more exposed during reconvergence	conntrack entries, TCP retransmits, CoreDNS p99, route-level latency, load-failure classes, recovery phases
Targeted network latency/loss	Colocation or dependency-aware placement reduces user-visible harm when a degraded edge is traversed less often.	Placement strategy; latency or loss level	Route p95/p99, error rate, retries, cart/checkout failures, east-west latency	Fixed destination hosts or IPs, fixed load, same pods targeted, same duration	Structural locality changes how often requests hit the impaired cross-node edge	route-level stats, latency prober, Redis/cart timings, cross-node variance, probe results
Node-cpu-hog	Spread reduces user-visible degradation and throttling relative to colocate under node-level CPU contention.	Placement strategy	route p95/p99, error rate, CPU throttling, CPU pressure, probe pass rate	Same load, duration, affinity, node selected via target service	Node-level CPU stress competes with colocated services for core time	CPU throttling, PSI CPU, pod/container CPU usage, Locust route stats

Falsifiable hypotheses for the top three designs

Hypothesis ID	Falsifiable statement
H-memory	Under fixed load and fixed node-memory-hog intensity, <code>colocate</code> will have a higher median route p99 latency and more OOM/eviction events than <code>spread</code> ; if those differences are absent or reversed with non-negligible effect size, the contention-based placement hypothesis is not supported for this workload.
H-churn	Under repeated 100% <code>pod-delete</code> of single-replica <code>productcatalogservice</code> , <code>spread</code> will show a larger during-chaos decrease in <code>contrack_entries_per_node</code> and higher transport-level disruption than <code>colocate</code> ; if not, the churn/reconvergence explanation is weakened.
H-network	Under destination-scoped network latency or loss on a high-traffic dependency edge, <code>dependency-aware</code> or <code>colocate</code> will produce lower user-visible p95/p99 latency than <code>spread</code> ; if route latency and error effects do not separate by placement, structural locality is not the dominant factor.

Which fault is most likely to yield the best result

This is the blunt answer you asked for.

Fault	Academic value	Likely placement sensitivity	My advice
Node-memory-hog	Very high	High	Run this. Best new main experiment.
Targeted network latency/loss	High	High if edge-scoped well	Run this if you have extra time.
Pod-delete churn	High, but methodological/mechanistic	Medium on score, high on mechanism	Keep as current core evidence.
Redis latency	Medium	Medium if dependency-topology-aware	Only if you can target caller-callee topology cleanly
Node-cpu-hog	Medium	Medium	Good backup contention experiment
Pod-cpu-hog	Low-medium	Weak / confounded	Do not make this a headline
Disk I/O stress	Low	Low on this workload	Drop it

The reason node-memory-hog wins is simple. Kubernetes documents that memory limits are reactive, node memory pressure can trigger eviction, and memory pressure is surfaced as a node condition and taint. That means a **node-scoped** memory fault acts on the exact layer where placement matters. By contrast, pod-level CPU stress often collapses into container-local throttling because CPU limits are hard-enforced by throttling. On a small cluster with 4 GiB workers, memory pressure is far more likely to generate a clean placement-sensitive result than pod-cpu-hog. ¹⁷

Literature claim to experiment mapping

This is the table you can almost lift into the thesis.

Literature claim	Experiment that tests or builds on it	Expected result	Interpretation if result differs
Spreading across failure domains improves availability and utilization	Compare <code>spread</code> vs <code>colocate</code> under <code>pod-delete</code> and <code>node-memory-hog</code>	Spread helps under node contention; not necessarily under pod-delete churn	If spread always wins, your churn story is too strong; if spread never wins, either locality dominates or your workload is too small/ idiosyncratic
Denser packing increases contention and hurts QoS	Compare placements under <code>node-memory-hog</code> or <code>node-cpu-hog</code>	Colocate / best-fit should look worse than spread	If not observed, either limits/QoS settings neutralized contention or workload is not contention-sensitive
Kubernetes Service routing converges asynchronously to endpoint changes	Measure pod-delete against conntrack / transport / route effects	Reconvergence signatures appear even when recovery time alone is inconclusive	If absent, you need to weaken the churn-mechanism claim substantially
Mechanism and user-visible metrics can decouple	Compare mechanism metrics, route-level outcomes, and aggregate score across faults	Partial decoupling, especially under pod-delete	If everything moves together cleanly, your measurement-layer thesis is weaker but still publishable as a positive alignment result
Mean-like aggregate metrics hide what matters at the tail	Compare mean score against route p95/p99 and lower-tail score variants	Rankings disagree or tails tell a clearer story	If all metrics agree, do not force a “score instability” narrative

Claims, threats, and defense strategy

The big overclaim risk is already visible in the repository's prose. The slide script says the literature-derived hypotheses are "refuted," that recovery time can "predict nothing," and that "the literature's spread is safer prescription is refuted at the metric level." Your own evidence base, as publicly described, is narrower than that. It supports a much safer claim: **for pod-delete on this cluster and workload, contention-based placement intuitions were not borne out by the aggregate score, while mechanism metrics suggested a different failure mechanism.** That is strong enough. It is also much harder to attack. 18

Keep, weaken, remove, future-work

Claim status	Recommendation
Keep	"ChaosProbe is a placement-aware chaos-evaluation framework with controlled placement mutation, cross-layer measurement, and statistical tooling."
Keep	"Aggregate resilience scores were unstable in the current churn campaign and should not be used alone to rank strategies."
Keep	"Mechanism metrics can be more reproducible than aggregate scores."
Weaken	"Pod-delete is a churn-based fault" → say "pod-delete behaved <i>consistently with</i> a churn/reconvergence mechanism in this setup."
Weaken	"Spread is safer is refuted" → say "the usual spread intuition did not transfer cleanly to this fault class in this setup."
Weaken	"Recovery time predicts nothing" → say "recovery time decomposition was unstable and did not provide a stable ranking signal here."
Remove	Any universal "best strategy" claim across all placements and faults.
Remove	Any central claim resting on pod-cpu-hog until you rerun it cleanly and verify labels and limits.
Move to future work	Larger clusters, multi-replica services, other CNIs, other kube-proxy modes, production traffic, scheduler integration, and generalization beyond Online Boutique.

Threats to validity and how to defend them

Threat	Why it matters	Defense
Single-replica services	Pod-delete at 100% guarantees direct unavailability for the target	State clearly that the design isolates recovery and reconvergence behavior, not HA failover behavior
Small cluster external validity	4×4 GiB workers plus one control plane is not production scale	Scope claims to comparable small clusters and record environment metadata

Threat	Why it matters	Defense
CNI / kube-proxy mode dependence	Calico, containerd, iptables/nftables details may matter	Archive CNI hint, kube-proxy mode, and kernel settings; present mechanism claims as environment-contingent
Metric portability	PSI, Felix, or some kube-proxy metrics may not be collected	Use <code>metricAvailability</code> ; never imply zeros where metrics were absent
Underpowered comparisons	n<8 can produce noise and false confidence	Reduce strategy count; increase valid iterations; lead with effect sizes and CIs
Load drift across runs	Different offered load can fake placement effects	Use fixed Locust profile and archive offered-RPS summaries; the repo already checks >20% drift
Placement mismatch	Scheduler overrides could invalidate intended placement	Treat match rate <1.0 as a quality problem, not a footnote
Evidence-chain gaps	Missing archives or mislabeled scenarios destroy credibility	Archive immutable raw outputs, scenario hashes, run manifests, and code commit IDs before submission

Exact evidence required for major claims

Claim	Required data	Required test	Required figure or table	Likely examiner objection	Concise defense
Aggregate score is unstable	Per-iteration scores across repeated runs and batches	Bootstrap CIs; variance/range summary; pairwise Holm-corrected comparisons	Raincloud/box plot of per-iteration scores by batch and strategy	"Maybe you just don't have enough samples."	"Correct; that is exactly why the score is not a reliable ranking metric at this scale."
Pod-delete is consistent with churn/reconvergence	Conntrack, retransmits, CoreDNS or proxy metrics, route tails	Pairwise comparison on churn metrics; temporal alignment around fault windows	Time-series around kill cycle plus aggregate effect-size table	"This could just be generic disruption."	"The transport/network metrics move in the predicted direction while pure contention metrics do not."

Claim	Required data	Required test	Required figure or table	Likely examiner objection	Concise defense
Node-memory-hog reveals placement-sensitive contention	OOMs, evictions, MemoryPressure, route latency, error rate	Pairwise or multi-factor nonparametric tests with effect sizes	Strategy × metric summary table and route-tail plots	"You forced a catastrophic node fault."	"We tuned to a sub-catastrophic regime and report the pilot sweep."
Mechanism metrics and user-visible outcomes decouple	Same-run mechanism metrics plus route-level outcomes	Correlation and rank-agreement analysis; not just mean comparison	Scatter or rank-order disagreement matrix	"Then why trust either?"	"Because different layers answer different questions; we do not collapse them prematurely."
Pod-cpu-hog is weak/inconclusive	CPU usage, throttling, per-pod utilization fractions, route stats	Descriptive comparison plus effect sizes	Appendix: self-throttle vs neighbor-throttle figure	"Convenient excuse for a null result."	"It is a falsifiable pilot; node-level contention is the cleaner experiment."
ChaosProbe is reproducible as an artifact	Raw archives, run metadata, code commit, scenario hashes, seeds	Re-run same analyses from archived bundles	Reproducibility appendix and artifact manifest	"This is just a custom script pile."	"No: it has tests, structured outputs, a quality checker, and a documented rerun path."

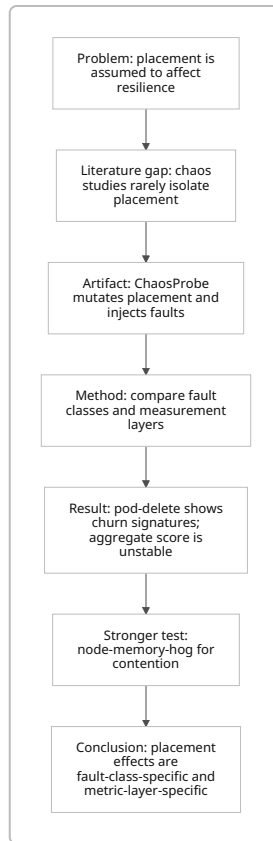
Hostile examiner questions and answer strategies

Hostile question	Answer strategy
"Why isn't this just a tooling project?"	"Because the artifact enables a substantive empirical finding: placement effects differ by fault class and by measurement layer."
"Did you prove spread is bad?"	"No. I showed that spread was not universally better under the pod-delete fault class in this setup."

Hostile question	Answer strategy
"Why should I care about single-replica pod-delete?"	"Because it isolates restart and service-network reconvergence behavior cleanly; I scope the conclusion to that regime."
"Why not use Chaos Mesh instead of Litmus?"	"The fault injector is not the contribution. Controlled placement mutation and comparative measurement are."
"Where is the causal evidence for contrack?"	"Direct causality is limited; I claim mechanistic consistency supported by Kubernetes networking semantics and layered measurements."
"Why only three strategies in the core result?"	"Because statistical power matters more than breadth for a defensible thesis."
"Why Mann-Whitney and not ANOVA?"	"Small n, non-normal distributions, and heavy-tailed outcomes; I complement p-values with effect sizes and bootstrap intervals."
"Why Neo4j?"	"It preserves topology and event relationships for analysis, but I do not oversell it as the thesis' main novelty."
"Could these results be specific to Calico or kube-proxy mode?"	"Yes, and I state that explicitly and archive the environment so the claim is scoped rather than inflated."
"Why should I trust a negative result about the score?"	"Because the baseline control holds, variance is quantified, and the repo's own analysis tooling warns against underpowered or tainted interpretation."

Thesis blueprint and completion plan

The right narrative arc is simple: **placement matters in theory; chaos tools can inject faults but usually do not isolate placement effects; Kubernetes faults act through heterogeneous mechanisms; ChaosProbe lets you manipulate placement and observe multiple layers; the resulting evidence shows fault-class-specific placement effects and warns against over-trusting aggregate scores.** That is clean, exam-proof, and much stronger than a messy "best strategy" pitch. ¹⁹



Thesis blueprint

Section	What it should do
Abstract	State the problem, the artifact, the manipulated variable, the fault classes, the key result, and the scope limit
Introduction	Motivate why placement is assumed to affect resilience, and why that assumption is too coarse without fault-class separation
Background	Briefly cover Kubernetes scheduling, Service/EndpointSlice routing, chaos engineering, and multi-layer resilience measurement
System design	Describe ChaosProbe architecture, placement engine, fault runner, metrics, storage, statistics, and provenance
Methodology	Define workload, cluster, strategies, faults, controls, repetitions, metrics, exclusion criteria, and statistics
Related work	Position against placement literature, chaos platforms, Kubernetes-failure studies, and measurement papers
Experiments	Present pod-delete and node-memory-hog as the core pair; keep network experiment as extension if time permits

Section	What it should do
Results	Lead with quality checks, then mechanism metrics, then user-visible metrics, then aggregate score, then interpretation
Discussion	Explain why the results are fault-class-specific, what the score instability means, and where generalization stops
Limitations	Single replicas, small cluster, CNI/proxy dependence, metric portability, limited fault catalog
Future work	Multi-replica, larger clusters, alternate CNIs/proxy modes, targeted network faults, scheduler integration

A concise abstract you can actually use:

Abstract. Pod placement in Kubernetes is widely assumed to affect service resilience, yet most chaos-engineering studies vary faults without systematically varying placement, and most placement studies do not test their assumptions under controlled fault injection. This thesis presents **ChaosProbe**, a placement-aware chaos-evaluation framework for Kubernetes that mutates pod placement strategies, executes LitmusChaos faults, collects cross-layer metrics from Kubernetes, Prometheus, Locust, and Litmus, and stores structured results for comparative analysis. Using the Online Boutique microservice application on a small Kubernetes cluster, the study compares placement strategies under multiple fault classes. The central result is methodological and mechanistic: under repeated `pod-delete` faults, aggregate resilience scores are too unstable to provide a reliable ranking of strategies, while mechanism-level metrics show reproducible differences consistent with service-network churn and control-plane reconvergence rather than resource contention. This motivates a fault-class-specific analysis of placement, and a second experiment design based on node-level memory pressure is proposed to test contention-sensitive behavior directly. The thesis argues that placement effects under chaos are both **fault-class-specific** and **measurement-layer-specific**, and that aggregate resilience scores should not be interpreted without mechanism and user-visible metrics.

Final figures and tables you should include

Figure or table	Caption goal
Architecture figure	"ChaosProbe workflow from placement mutation to fault execution, data collection, statistics, and graph storage." Proves this is a real system, not scripts glued together.
Experimental design table	"Cluster, workload, strategies, faults, controls, repetitions, and analyzed metrics." Proves fairness and rigor.
Quality-control figure	"Baseline control and data-quality filters." Proves that noisy conclusions are not due to obvious instrumentation failure.

Figure or table	Caption goal
Pod-delete time-series figure	"Transport or proxy-adjacent metrics around repeated deletion cycles." Proves the churn story mechanistically.
Recovery split figure	"Deletion-to-scheduled versus scheduled-to-ready across strategies." Proves whether scheduler stall or startup dominates.
User-visible route-tail figure	"Route p95/p99 and error rate by strategy and fault." Proves you care about users, not only infrastructure.
Aggregate score instability figure	"Same strategy across repeated runs." Proves why the score is not a robust ranking metric.
Node-memory-hog figure	"Memory pressure, OOM/eviction counts, and route tails by placement." Proves contention-sensitive placement effect.
Claim evidence matrix	"Which claims rest on which data and tests." Proves discipline and helps examiners.
Threats-to-validity table	"Internal, construct, external, and conclusion validity threats and mitigations." Proves honesty.

Reproducibility package requirements

Archive these for **every reported run**:

Must archive	Why
<code>summary.json</code> per run	Main analysis input
Raw Locust CSVs and failure logs	User-visible outcome ground truth
Raw Prometheus query outputs or snapshotted exports	Mechanism verification
LitmusChaos experiment manifests and logs	Fault provenance
Exact scenario YAMLs and SHA-256 hashes	Prevent silent scenario drift
Git commit hash and dirty/clean flag	Code provenance
Cluster metadata: K8s version, kube-proxy mode, container runtime, CNI, node sizes	Scope and rerun fidelity
Random seeds and seed-expansion policy	Reproducibility of randomized placement
Batch/day identifier	Separate run-to-run cluster drift from strategy effects
Figure-generation scripts	Reviewer rerun path
A single artifact manifest	One file examiners can read first

Actionable code, experiment, and documentation changes

Area	Change
Code	Record kube-proxy mode and relevant kernel/conntrack settings in <code>runMetadata</code>
Code	Add EndpointSlice snapshots and service endpoint counts around fault windows
Code	Persist scenario hashes automatically and fail if a run lacks them
Code	Make batch/day ID first-class metadata for mixed-run analysis
Experiments	Add a pilot sweep script for tuning node-memory-hog intensity before the main campaign
Experiments	Reduce the main matrix to three core strategies; keep others in appendix
Experiments	Stop using pod-cpu-hog as a headline unless you first prove it stresses neighbors rather than only the target
Documentation	Add a public immutable <code>thesis-results/</code> manifest or release bundle if it is not already exposed
Documentation	Add an explicit “scope of claims” page saying what generalizes and what does not
Claims	Replace every “refuted,” “proven,” “best strategy,” and “reproducible” with scoped versions unless backed by archived evidence

Completion checklist

Must-have before submission

- Archived raw data and immutable result manifest for every figure-backed claim.
- Final core experiments: pod-delete and node-memory-hog.
- At least 8 valid iterations per strategy per core fault.
- Public or appendix-visible evidence matrix connecting claims to data and tests.
- Clear statement of scope: single-replica Online Boutique, small cluster, specific K8s environment.
- Clean thesis figures for mechanism metrics, user-visible metrics, and score instability.
- Reproducibility appendix with commit hashes, scenario hashes, seeds, and environment details.
- All overclaims removed from manuscript and slides.

Should-have for a strong defense

- One targeted network-latency or network-loss experiment.
- Route-level breakdown rather than only whole-application aggregates.
- A compact rerun script that regenerates all main tables and figures.
- One appendix table showing placement match rates and excluded/tainted iterations.
- One page explaining why pod-cpu-hog was demoted or how it was repaired.

Nice-to-have if time remains

- One multi-replica extension.
- One mixed-effects or aligned-rank interaction analysis.
- A release artifact or Zenodo-like archive.
- A stronger dependency-aware placement evaluation.

Unsafe claims to remove

- "The literature is refuted."
- "Spread is worse."
- "We proved causality."
- "This generalizes to Kubernetes clusters broadly."
- "Resilience score identifies the best strategy."
- "The result is reproducible" unless you mean the specific mechanism metrics and can show the rerun package.

Minimal viable thesis plan

If time is tight, this is the smallest plan I would still defend:

- Artifact chapter on ChaosProbe.
- Core strategies: `default`, `colocate`, `spread`.
- Core faults: `pod-delete`, `node-memory-hog`.
- Core claims:
 - the score is unstable and not enough on its own;
 - pod-delete produces a churn-consistent mechanism story;
 - node-memory-hog yields a clearer contention-sensitive placement effect.
- Core figures:
 - architecture;
 - quality control;
 - pod-delete mechanism time series;
 - route-tail comparison;
 - aggregate score instability;
 - node-memory-hog placement effect.
- Core tables:
 - methodology;
 - related work positioning;
 - threats to validity;
 - evidence matrix.

That is enough for a credible MSc thesis.

Stronger ambitious thesis plan

If you have more time, add:

- `dependency-aware` as a fourth strategy.

- One destination-scoped network latency or loss experiment.
- One limited multi-replica extension.
- A stronger interaction analysis across strategy × fault class.
- A discussion chapter on how placement guidance should be conditional on dominant fault class and objective metric.

A one-page defense summary you can memorize

What problem did I study?

Placement is often assumed to affect resilience, but the literature and existing chaos platforms do not cleanly tell us how placement interacts with different fault classes in Kubernetes. ²⁰

What did I build?

I built ChaosProbe, a placement-aware chaos-evaluation framework. It mutates pod placements, runs LitmusChaos faults, collects Kubernetes, Prometheus, Locust, and Litmus metrics, stores structured results, and compares strategies statistically with provenance and data-quality checks. ²¹

What is the main scientific question?

How do placement strategies interact with fault class, and which measurement layers provide stable evidence of resilience differences? ²²

What is the main result?

In the current pod-delete study, aggregate resilience scores are too unstable to rank strategies reliably, but mechanism-level metrics provide a more reproducible signal. That means aggregate scores alone can be misleading. ²³

What mechanism explains the pod-delete behavior?

The evidence is most consistent with service-network churn and reconvergence around EndpointSlice, kube-proxy, and conntrack state, not just simple co-location contention. Kubernetes' own networking model makes that plausible. ²⁴

What is the next best experiment?

Node-memory-hog, because it introduces genuine node-scoped contention where placement should matter strongly, unlike pod-cpu-hog which is often confounded by CPU-limit throttling. ²⁵

What do I claim and what do I not claim?

I claim that placement effects are fault-class-specific and measurement-layer-specific in this experimental regime. I do **not** claim a universal best strategy or broad generalization to all Kubernetes deployments. ²⁶

Why is this an MSc thesis and not just engineering?

Because the artifact enables a scoped empirical contribution: it shows how a widely held placement intuition changes once fault class and measurement layer are treated explicitly, and it does so with a reproducible experimental framework. ²⁷

Open questions and limitations

I reviewed the publicly accessible repository pages, technical docs, slide script, and related references, but I did **not** inspect immutable archived raw run bundles for the thesis results in the public paths I reviewed. The strongest final claims therefore still depend on whether you can produce and archive those raw artifacts cleanly at submission time. Also, the current public evidence is strongest for the pod-delete mechanism story and much weaker for pod-cpu-hog as a general contention result. If you do nothing else, fix that asymmetry with a clean node-memory-hog campaign and an artifact-grade result archive. ²⁸

¹ ²⁸ <https://github.com/yhu02/uva-msc-thesis>

<https://github.com/yhu02/uva-msc-thesis>

² ⁷ ¹³ ¹⁹ ²⁰ <https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>

<https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>

³ ²⁶ https://github.com/yhu02/uva-msc-thesis/blob/main/presentation_script.md

https://github.com/yhu02/uva-msc-thesis/blob/main/presentation_script.md

⁴ <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>

<https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>

⁵ <https://hub.litmuschaos.io/api/chaos/master?file=faults%2Fkubernetes%2Fnode-memory-hog%2Ffault.yaml>

<https://hub.litmuschaos.io/api/chaos/master?file=faults%2Fkubernetes%2Fnode-memory-hog%2Ffault.yaml>

⁶ ⁸ ¹⁵ ¹⁶ ²¹ ²⁷ [uva-msc-thesis/chaosprobe/TECHNICAL.md at main · yhu02/uva-msc-thesis · GitHub](#)

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/TECHNICAL.md>

⁹ <https://research.google/pubs/borg-omega-and-kubernetes/>

<https://research.google/pubs/borg-omega-and-kubernetes/>

¹⁰ ¹⁸ <https://github.com/yhu02/uva-msc-thesis/blob/main/references.md>

<https://github.com/yhu02/uva-msc-thesis/blob/main/references.md>

¹¹ <https://arxiv.org/abs/2404.11169>

<https://arxiv.org/abs/2404.11169>

¹² <https://github.com/googlecloudplatform/microservices-demo>

<https://github.com/googlecloudplatform/microservices-demo>

¹⁴ <https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/how-to/run-experiments.md>

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/how-to/run-experiments.md>

¹⁷ ²⁵ <https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>

<https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>

²² ²⁴ <https://kubernetes.io/docs/reference/networking/virtual-ips/>

<https://kubernetes.io/docs/reference/networking/virtual-ips/>

²³ [uva-msc-thesis/chaosprobe/docs/how-to/reproducing-thesis-results.md at main · yhu02/uva-msc-thesis · GitHub](#)

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/how-to/reproducing-thesis-results.md>